条形码识别

条形码识别
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构
- find_barcodes([roi])
- `BarCode` 类
  - 构造函数
  - `corners`
  - `rect`
  - `x`
  - `y`
  - `w`
  - `h`
  - `payload`
  - `type`
  - `rotation`
  - `quality`

例程实验效果简介

这一节我们来用K230识别一下条形码，首先我们用条形码生成器来生成一个条形码图片

（可以使用我们附件里提供的条形码生成工具，也可以在网上搜素在线生成条形码）

在输入框里输入英文或数字内容，点击生成条形码按钮即可生成



我们把生成的条形码放大

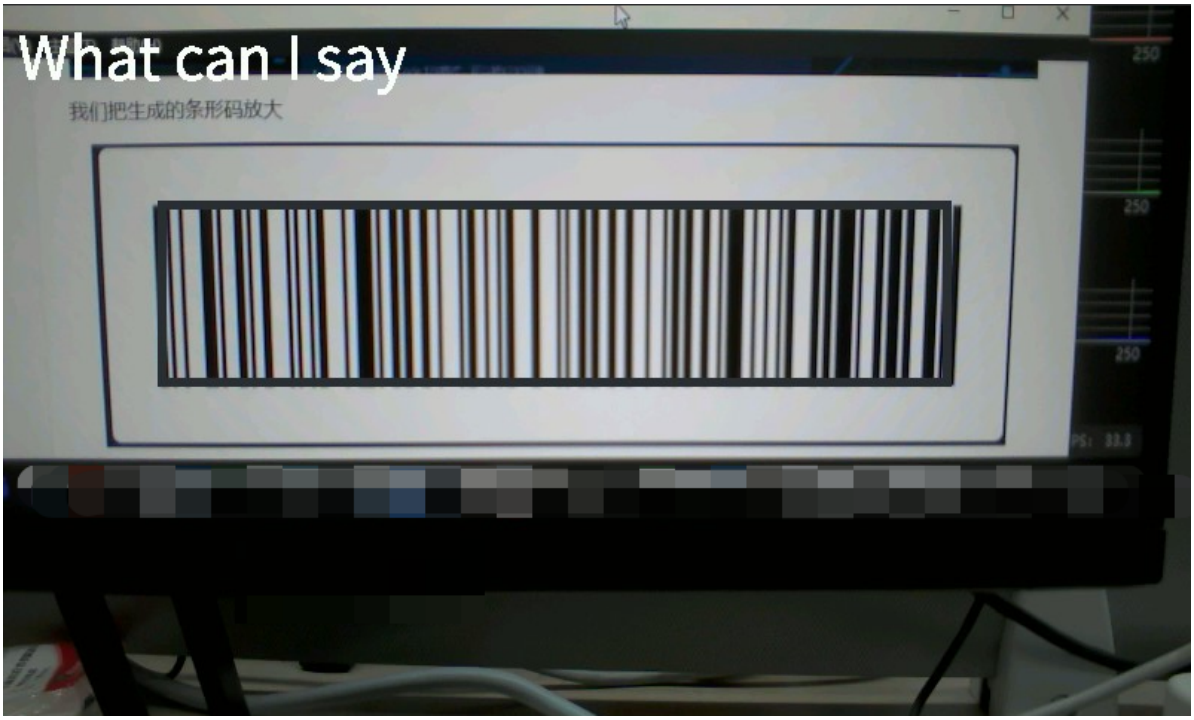


本节K230部分的例程代码位于：【源码汇总 / 06.Codes / 01.find_barcodes.py】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

点击CanMV IDE左下角的运行按钮运行例程

用摄像头对准刚刚生成的二维码，可以看到成功识别到了条形码的内容



同时CanMV IDE的左下角也会输出我们的识别结果

```
Barcode CODE128, Payload "What can I say"
Barcode CODE128, Payload "What can I say"
Barcode CODE128, Payload "What can I say"
Barcode CODE128, Payload "What can I say"
Barcode CODE128, Payload "What can I say"
Barcode CODE128, Payload "What can I say"
Barcode CODE128, Payload "What can I say"
Barcode CODE128, Payload "What can I say"
Barcode CODE128, Payload "What can I say"
```

代码讲解

本节我们要用的的外设主要是摄像头模块

条形码的检测和识别 由 K230中的 find_barcodes() 方法实现，该方法属于image模块

完整代码

```
# 导入必要的模块 Import required modules
import time
import gc
from media.sensor import Sensor
from media.display import Display
from media.media import MediaManager
import image

# 定义条形码类型映射字典 Define barcode type mapping dictionary
BARCODE_TYPES = {
    image.EAN2: "EAN2",
    image.EAN5: "EAN5",
    image.EAN8: "EAN8",
    image.UPCE: "UPCE",
    image.ISBN10: "ISBN10",
    image.UPCA: "UPCA",
    image.EAN13: "EAN13",
    image.ISBN13: "ISBN13",
    image.I25: "I25",
    image.DATABAR: "DATABAR",
    image.DATABAR_EXP: "DATABAR_EXP",
    image.CODABAR: "CODABAR",
    image.CODE39: "CODE39",
    image.PDF417: "PDF417",
    image.CODE93: "CODE93",
    image.CODE128: "CODE128"
}

def barcode_name(code):
    """
    获取条形码类型名称
    Get barcode type name
    """
    return BARCODE_TYPES.get(code.type(), "UNKNOWN")

def init_camera():
    """
    初始化摄像头设置
    Initialize camera settings
    """
    sensor = Sensor()  # 构建摄像头对象 Create camera object
    sensor.reset()     # 复位和初始化摄像头 Reset and initialize camera
    # 设置帧大小为LCD分辨率(640x480) Set frame size to LCD resolution (640x480)
    sensor.set_framesize(width=640, height=480)
    # 设置输出图像格式 Set output image format
    sensor.set_pixformat(Sensor.RGB565)
    return sensor

def main():
    # 初始化摄像头 Initialize camera
    sensor = init_camera()

    # 初始化显示设置 Initialize display settings
    Display.init(Display.ST7701, to_ide=True)
```

```
    # 初始化media资源管理器 Initialize media resource manager
    MediaManager.init()

    # 启动sensor Start the sensor
    sensor.run()

    # 创建时钟对象用于FPS计算 Create clock object for FPS calculation
    clock = time.clock()

    while True:
        clock.tick()

        # 捕获图像 Capture image
        img = sensor.snapshot()

        # 查找图像中所有条形码 Find all barcodes in the image
        codes = img.find_barcodes()

        for code in codes:
            # 用矩形框标记条码位置 Mark barcode position with rectangle
            img.draw_rectangle(code.rect(), thickness=6, color=(46, 47, 48))

            # 获取条码类型和内容 Get barcode type and content
            code_type = barcode_name(code)
            payload = code.payload()

            # 打印条码信息 Print barcode information
            print(f"Barcode {code_type}, Payload \"{payload}\"")

            # 在图像中显示条码内容 Display barcode content in image
            img.draw_string_advanced(10, 10, 40, payload, color=(255, 255, 255))

        # 显示处理后的图像 Display processed image
        Display.show_image(img)

        # 执行垃圾回收 Perform garbage collection
        gc.collect()

if __name__ == "__main__":
    main()
```

代码结构

本节例程主要功能和结构如下：

1. 模块导入部分：
    - 导入了时间处理、垃圾回收、传感器、显示器、媒体管理和图像处理等必要模块
2. 条形码类型定义：
    - 创建了一个字典`BARCODE_TYPES`，映射了各种条形码类型（如EAN13、ISBN13、CODE128等）
3. 主要函数：
    - `barcode_name()`：获取条形码类型的名称
    - `init_camera()`：初始化摄像头，设置分辨率(800x480)和图像格式

- `main()`: 程序的主要运行逻辑

4. 主程序流程：
    - 初始化摄像头、显示器和媒体管理器
    - 进入无限循环，持续执行：
        - 捕获图像
        - 在图像中查找条形码
        - 对每个找到的条形码：
            - 用矩形框标记位置
            - 获取并显示条形码类型和内容
        - 显示处理后的图像
        - 进行垃圾回收

find_barcodes([roi])

```
image.find_barcodes([roi])
```

该函数查找指定 ROI 内的所有一维条形码，并返回一个包含 `image.barcode` 对象的列表。有关更多信息，请参考 `image.barcode` 对象的相关文档。

为了获得最佳效果，建议使用长为 640 像素、宽为 40/80/160 像素的窗口。窗口的垂直程度越低，运行速度越快。由于条形码是线性一维图像，因此在一个方向上需具有较高分辨率，而在另一个方向上可具有较低分辨率。请注意，该函数会进行水平和垂直扫描，因此您可以使用宽为 40/80/160 像素、长为 480 像素的窗口。请务必调整镜头，使条形码位于焦距最清晰的区域。模糊的条形码无法解码。

该函数支持以下所有一维条形码：

- `image.EAN2`
- `image.EAN5`
- `image.EAN8`
- `image.UPCE`
- `image.ISBN10`
- `image.UPCA`
- `image.EAN13`
- `image.ISBN13`
- `image.I25`
- `image.DATABAR (RSS-14)`
- `image.DATABAR_EXP (RSS-Expanded)`
- `image.CODABAR`
- `image.CODE39`
- `image.PDF417`
- `image.CODE93`
- `image.CODE128`
- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。

注意： 不支持压缩图像和 Bayer 图像。

`BarCode`类

条形码对象由 `image.find_barcodes` 函数返回。

构造函数

```
class image.barcode
```

请调用 `image.find_barcodes()` 函数来创建此对象。

`corners`

```
barcode.corners()
```

该方法返回一个包含条形码四个角的元组列表，每个元组格式为 (x, y)。四个角通常按从左上角开始，沿顺时针方向排列。

`rect`

```
barcode.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于其他图像处理方法，如 `image.draw_rectangle` 中的条形码边界框。

`x`

```
barcode.x()
```

该方法返回条形码边界框的 x 坐标（整数）。您也可以通过索引 [0] 获取该值。

`y`

```
barcode.y()
```

该方法返回条形码边界框的 y 坐标（整数）。您也可以通过索引 [1] 获取该值。

`w`

```
barcode.w()
```

该方法返回条形码边界框的宽度（整数）。您也可以通过索引 [2] 获取该值。

`h`

```
barcode.h()
```

该方法返回条形码边界框的高度（整数）。您也可以通过索引 [3] 获取该值。

payload

```
barcode.payload()
```

该方法返回条形码的有效载荷字符串，例如：“数量”。

您也可以通过索引 [4] 获取该值。

type

```
barcode.type()
```

该方法返回条形码的类型（整数）。可能的类型包括：

- image.EAN2
- image

.EAN5

- image.EAN8
- image.UPCE
- image.ISBN10
- image.UPCA
- image.EAN13
- image.ISBN13
- image.I25
- image.DATABAR
- image.DATABAR_EXP
- image.CODABAR
- image.CODE39
- image.PDF417（未来启用，当前尚不可用）
- image.CODE93
- image.CODE128

您也可以通过索引 [5] 获取该值。

rotation

```
barcode.rotation()
```

该方法返回条形码的旋转角度（以弧度计量，浮点数）。

您也可以通过索引 [6] 获取该值。

quality

```
barcode.quality()
```

该方法返回条形码在图像中被检测到的次数（整数）。

在扫描条形码时，每一条新的扫描线都能解码相同的条形码。每次进行此过程时，条形码的值都会增加。

您也可以通过索引 [7] 获取该值。